

作业 2 运算符重载

提交要求:

1. 提交一个 word 或 pdf 文件
2. 填空题给出完整代码，填空部分的代码黄色高亮显示。
3. 程序设计题目给出完整代码，以及运行结果的截图。

一 填空

1. 完善以下程序。

```
#include <iostream>
using namespace std;

class Temp{
    double ce;
public:
    Temp(double c = 0) : ce(c) {}
    double getCe() const { return ce; }

    // 重载 +=
    Temp& operator+=(const Temp& other) {
        ce += other.ce;
        return _____; // 填空(1)
    }

    // 重载 + (友元)
    friend Temp operator+(const Temp& a, const Temp& b);

    // 前置++
    Temp& operator++() {
        ++ce;
        return _____; // 填空(2)
    }

    // 后置++
    Temp operator++(int) {
        Temp temp = *this;
        ++(*this);
        return _____; // 填空(3)
    }
};

Temp operator+(const Temp& a, const Temp& b) {
    return Temp(a.ce + b.ce);
}

int main() {
    Temp t1(10), t2(5);
    t1 += t2;
    Temp t3 = t1 + t2;
```

```

    cout << t3.getCe() << endl;    // 输出 20
    ++t1;
    cout << t1.getCe() << endl;    // 输出 16
    Temp t4 = t1++;
    cout << t4.getCe() << endl;    // 输出 16
    cout << t1.getCe() << endl;    // 输出 17
    return 0;
}

```

二 程序设计

1. 设计一个**三维向量类** Vector3D, 包含:

- 私有数据成员: x, y, z (double)。
- 构造函数和重载 <<。
- 重载 + 和 - (成员函数, 返回新对象)。
- 重载 * 实现两种功能:
 - 向量与标量相乘 (Vector3D * double, 成员函数)
 - 标量与向量相乘 (double * Vector3D, 友元函数实现交换律)
- 重载 == 和 != (友元函数, 判断分量是否相等)

main 函数及其输入如下:

```

Vector3D v1(1, 2, 3), v2(4, 5, 6);
Vector3D v3 = v1 + v2;
Vector3D v4 = v1 - v2;
Vector3D v5 = v1 * 2.5;
Vector3D v6 = 3.0 * v2;
cout << "v1+v2: " << v3 << endl;    //v1+v2: (5,7,9)
cout << "v1-v2: " << v4 << endl;    //v1-v2: (-3,-3,-3)
cout << "v1*2.5: " << v5 << endl;    //v1*2.5: (2.5,5,7.5)
cout << "3*v2: " << v6 << endl;    //3*v2: (12,15,18)
cout << "(v1==v2): " << (v1 == v2) << endl;    //(v1==v2): 0

```

2. 设计时间类 STime, 表示一天中的某个时刻 (时、分、秒)。要求:

- 私有数据成员:
 - int hour, minute, second
 - static int defaultFormat: 静态成员, 默认为 0 (表示 12 小时制), 为 1 时表示 24 小时制 (影响输出格式)
- 公有成员函数:
 - 构造函数: Time(int h = 0, int m = 0, int s = 0), 若小时、分钟、秒超出范围(0-23,0-59,0-59) 则设为合法默认值。
 - void normalize(): 规范化时间 (处理进位和负值, 例如秒≥60 则进位到分, 如果秒<0 则向分借位)。
 - 前置 --: 将当前时间减少 1 秒 (考虑借位), 返回引用。
 - 后置 --: 将当前时间减少 1 秒, 返回减少前的副本。

- 复合赋值 `operator-= (int sec)`: 减少指定秒数（秒数可能大于当前的 `second` 数值），返回引用。
- 类型转换运算符 `operator int() const`: 返回从 00:00:00 到当前时刻的总秒数。
- `const` 成员函数 `int getTotalSec() const`: 返回总秒数（与类型转换功能相同，但作为显式成员）。
- 静态成员函数 `void setFormat(int fmt)`: 修改 `defaultFormat`。
- 友元 `ostream& operator<<(ostream&, const Time&)`: 根据 `defaultFormat` 输出时间。若为 12 小时制，输出 "hh:mm:ss AM/PM" (上午 0-11 为 AM, 12-23 为 PM, 注意 12 点显示 12:xx:xx PM); 若为 24 小时制，输出 "hh:mm:ss" (小时两位)。
- 所有成员函数在类外定义，正确使用 `static`、`const`。

主函数及其输出如下：

```
int main() {
    STime t1(23, 59, 59);
    cout << "t1: " << t1 << endl;           // 默认 12 小时制, 23:59:59 PM
    --t1;
    cout << "After --t1: " << t1 << endl;     // 23:59:58 PM

    STime t2(1, 0, 0);
    STime t3 = t2--;                          // 后置--
    cout << "t3 (old t2): " << t3 << endl;    // 01:00:00 AM
    cout << "t2 now: " << t2 << endl;        // 00:59:59 AM

    t1 -= 7200;                               // 减 2 小时
    cout << "t1 after -=7200: " << t1 << endl;

    int total = t1;                           // 类型转换
    cout << "Total seconds of t1: " << total << endl;
    Time::setFormat(1);                       // 改为 24 小时制
    cout << "t1 (24h): " << t1 << endl;

    return 0;
}
```

输出如下：

```
t1: 11:59:59 PM
After --t1: 11:59:58 PM
t3 (old t2): 01:00:00 AM
t2 now: 12:59:59 AM
t1 after -=7200: 09:59:58 PM
Total seconds of t1: 79198
t1 (24h): 21:59:58
```